Moments of Points, Line Segments, and Polygons
A Step-by-Step Derivation (Undergraduate-Friendly)
Chunk 1: Theory + Closed Forms (No Code Yet)

Contents

1 Why moments, and what we will derive **2**

2 What is a moment? **3**

3 The three bases (fully defined) **3**
- 3.1 Geometric (monomial) basis . . . 3
- 3.2 Fourier basis . . . 3
- 3.3 Legendre basis . . . 3

4 Shape models (point, segment, polygon) **4**
- 4.1 Point model . . . 4
- 4.2 Line segment model . . . 4
- 4.3 Polygon model . . . 4

5 Case group 1: Point moments (3 cases), fully derived **4**

6 Case group 2: Line segment moments (3 cases), fully derived **5**
- 6.1 Step 1: Reduce the 2D integral to a 1D integral . . . 5
- 6.2 Segment geometric moments: step-by-step to a finite sum . . . 6
- 6.3 Segment Fourier moments: step-by-step to a closed form . . . 7
- 6.4 Segment Legendre moments: step-by-step to a finite sum (no integrals left) . . . 9

7 Case group 3: Polygon moments via Green's theorem (very detailed) **10**
- 7.1 Green's theorem: the version we use . . . 10
- 7.2 How a polygon boundary integral becomes a sum over edges . . . 11
- 7.3 Parameterizing one edge and its outward normal (carefully) . . . 11
- 7.4 Polygon geometric moments: very detailed derivation to an edge formula . . . 12
- 7.5 Polygon Fourier moments: very detailed derivation to an edge formula . . . 13
- 7.6 Polygon Legendre moments: very detailed reduction to geometric moments . . . 15

8 Two polygon examples (conceptual setup for Chunk 2) **16**
- 8.1 Example A: Square centered at origin, side length 0.3 . . . 16
- 8.2 Example B: A less symmetric polygon . . . 17

9 Summary of the closed forms derived in Chunk 1 **17**

10 What this chunk contains **17**

11 Shared example inputs **18**
11.1 Two index pairs . . . 18
11.2 Point example . . . 18
11.3 Line segment example . . . 18
11.4 Polygon example A: square centered at origin, side length 0.3 . . . 18
11.5 Polygon example B: less symmetric quadrilateral . . . 18

12 Point results (3 cases) **18**
12.1 For $(m, n) = (2, 1)$ . . . 18
12.2 For $(m, n) = (2, 2)$ . . . 18

13 Line segment results (3 cases) **19**
13.1 For $(m, n) = (2, 1)$ . . . 19
13.2 For $(m, n) = (2, 2)$ . . . 19

14 Polygon A: square centered at origin, side 0.3 **19**
14.1 Why the square "collapses" to simpler formulas . . . 19
14.2 Square: geometric and Legendre "collapse" checks . . . 19
14.3 Square totals (boundary computation) . . . 20
14.4 Square per-edge tables . . . 20

15 Polygon B: less symmetric quadrilateral **20**
15.1 Quadrilateral totals . . . 20
15.2 Quadrilateral per-edge tables . . . 21

16 Python code (modular, reproduces all numbers and prints tables) **21**

1 Why moments, and what we will derive

Moments are a classic way to represent shapes. The idea is simple: choose a family of basis functions $\phi_{mn}(x, y)$ indexed by integers m, n, and compute how strongly the shape overlaps each basis function.

This document derives **closed-form** moment equations for three kinds of shapes:

point, **line segment**, **polygon (filled region)**.

And we do this for three popular bases:

Geometric (monomial), **Fourier (complex exponential)**, **Legendre (orthogonal polynomial)**.

That is $3 \times 3 = 9$ cases in total.

Important promise. Every boxed equation in this document is a true closed form:

- no remaining unevaluated integrals,
- only finite sums and elementary functions (powers, exponentials, factorials),
- and expressions that map directly to code.

Coordinate domain. We assume the geometry has already been normalized into $[-1,1]^2$. (If your original coordinates are different, you would typically apply an affine transform first.)

2 What is a moment?

Let $f(x,y)$ represent a shape in the plane. A **moment coefficient** C_{mn} is defined as:

$$C_{mn} = \int_{-1}^{1}\int_{-1}^{1} f(x,y)\,\phi_{mn}(x,y)\,dx\,dy. \tag{1}$$

Think of this as a dot product:

$$C_{mn} = \langle f, \phi_{mn}\rangle.$$

The indexing by (m,n) is like using different "frequencies" or "polynomial degrees." Low degrees capture coarse properties; high degrees capture fine details.

3 The three bases (fully defined)

We now define $\phi_{mn}(x,y)$ for each basis.

3.1 Geometric (monomial) basis

$$\phi^{\text{geo}}_{mn}(x,y) = x^m y^n. \tag{2}$$

These are the simplest moments. They are not orthogonal, but they are easy to compute.

3.2 Fourier basis

Let $j = \sqrt{-1}$ be the imaginary unit. Define:

$$\phi^{\text{F}}_{mn}(x,y) = e^{-j\pi(mx+ny)}. \tag{3}$$

These coefficients are generally complex. They are closely related to the 2D Fourier transform evaluated at integer frequencies.

3.3 Legendre basis

Legendre moments use the orthogonal Legendre polynomials $P_m(x)$ on $[-1,1]$. Define:

$$\phi^{\text{L}}_{mn}(x,y) = P_m(x)\,P_n(y). \tag{4}$$

We need an explicit polynomial formula for P_m. A standard closed form is:

$$P_m(x) = \sum_{k=0}^{\lfloor m/2\rfloor} A_{m,k}\,x^{m-2k}, \qquad A_{m,k} = (-1)^k\,\frac{(2m-2k)!}{2^m\,k!\,(m-k)!\,(m-2k)!}. \tag{5}$$

This tells you exactly what polynomial you are using, and it is easy to implement.

Quick sanity check. For $m=0$, $\lfloor m/2\rfloor = 0$, and the formula gives $P_0(x)=1$. For $m=1$, it gives $P_1(x)=x$. For $m=2$, it gives $P_2(x)=\frac{1}{2}(3x^2-1)$. So it matches the standard Legendre polynomials.

4 Shape models (point, segment, polygon)

The "same" moment definition (1) will apply to all shapes, but the key difference is how we represent $f(x, y)$.

4.1 Point model

A point at (x_0, y_0) is represented by Dirac deltas:

$$f_{\text{pt}}(x, y) = \delta(x - x_0)\,\delta(y - y_0). \tag{6}$$

You can think of this as "all mass concentrated at a single location."

4.2 Line segment model

Let the segment start at $a = (a_x, a_y)$ and end at $b = (b_x, b_y)$. Define the displacement vector and length:

$$d = b - a = (d_x, d_y), \qquad L = \|d\| = \sqrt{d_x^2 + d_y^2}.$$

A standard parameterization is:

$$x(t) = a_x + t d_x, \qquad y(t) = a_y + t d_y, \qquad t \in [0, 1].$$

We model the segment as an arclength-weighted ridge:

$$f_{\text{seg}}(x, y) = \int_0^1 \delta(x - x(t))\,\delta(y - y(t))\,L\,dt. \tag{7}$$

Why the factor L? Because t is not arclength; it runs from 0 to 1 regardless of how long the segment is. Arclength along a straight segment is:

$$ds = L\,dt.$$

So $L\,dt$ makes the measure "uniform per unit length," which is the most common convention.

4.3 Polygon model

Let $\Omega \subset \mathbb{R}^2$ be the filled region of a polygon. We use the indicator function:

$$f_{\text{poly}}(x, y) = \begin{cases} 1, & (x, y) \in \Omega, \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

Polygon moments are therefore **area integrals** over Ω.

5 Case group 1: Point moments (3 cases), fully derived

Start from (1) and plug in (6):

$$C_{mn}^{\text{pt}} = \iint \delta(x - x_0)\delta(y - y_0)\,\phi_{mn}(x, y)\,dx\,dy.$$

The key delta identity in one dimension is:

$$\int_{-\infty}^{\infty} \delta(x - x_0)\, g(x)\, dx = g(x_0).$$

We apply it twice:

- Integrate with respect to x: it replaces x by x_0.
- Integrate with respect to y: it replaces y by y_0.

So:

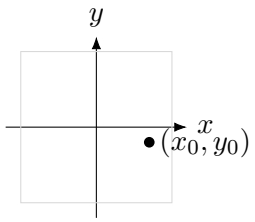
$$C_{mn}^{\text{pt}} = \phi_{mn}(x_0, y_0).$$

$$\boxed{C_{mn}^{\text{pt}} = \phi_{mn}(x_0, y_0).} \tag{9}$$

Now substitute each basis definition:

$$\boxed{\begin{aligned} C_{mn}^{\text{pt,geo}} &= x_0^m y_0^n, \\ C_{mn}^{\text{pt,F}} &= e^{-j\pi(mx_0+ny_0)}, \\ C_{mn}^{\text{pt,L}} &= P_m(x_0)\, P_n(y_0). \end{aligned}} \tag{10}$$

Figure (point in $[-1,1]^2$).



6 Case group 2: Line segment moments (3 cases), fully derived

This is the first place where students often feel lost, so we go slowly.

6.1 Step 1: Reduce the 2D integral to a 1D integral

Start with the moment definition:

$$C_{mn}^{\text{seg}} = \iint f_{\text{seg}}(x, y)\, \phi_{mn}(x, y)\, dx\, dy.$$

Substitute the segment function (7):

$$C_{mn}^{\text{seg}} = \iint \left(\int_0^1 \delta(x - x(t))\delta(y - y(t))\, L\, dt \right) \phi_{mn}(x, y)\, dx\, dy.$$

We now swap the order of integration. Intuitively: we want to integrate over t last, because it parameterizes the segment. Formally, since everything is well-behaved here, we can write:

$$C_{mn}^{\text{seg}} = \int_0^1 \left(\iint \delta(x - x(t))\delta(y - y(t))\, \phi_{mn}(x, y)\, dx\, dy \right) L\, dt.$$

Now focus on the inner 2D integral:

$$\iint \delta(x - x(t))\delta(y - y(t))\, \phi_{mn}(x, y)\, dx\, dy.$$

Interpretation:

- The $\delta(x - x(t))$ kills the x-integral and sets $x = x(t)$.
- The $\delta(y - y(t))$ kills the y-integral and sets $y = y(t)$.

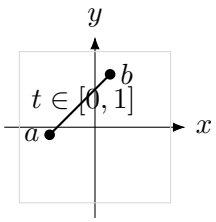
So the inner integral equals $\phi_{mn}(x(t), y(t))$.
Therefore:

$$C_{mn}^{\text{seg}} = \int_0^1 \phi_{mn}(x(t), y(t))\, L\, dt.$$

$$\boxed{C_{mn}^{\text{seg}} = L \int_0^1 \phi_{mn}(a_x + td_x,\ a_y + td_y)\, dt.} \tag{11}$$

This "master formula" is the starting point for all three segment bases.

Figure (segment in $[-1, 1]^2$).



6.2 Segment geometric moments: step-by-step to a finite sum

For geometric moments:

$$\phi_{mn}^{\text{geo}}(x, y) = x^m y^n.$$

Plug into (11):

$$C_{mn}^{\text{seg,geo}} = L \int_0^1 (a_x + td_x)^m (a_y + td_y)^n\, dt.$$

At this point, we use only one idea: **expand polynomials and integrate term-by-term**.

Step A: Expand $(a_x + td_x)^m$

By the binomial theorem:

$$(a_x + td_x)^m = \sum_{i=0}^{m} \binom{m}{i} a_x^{m-i} (td_x)^i = \sum_{i=0}^{m} \binom{m}{i} a_x^{m-i} d_x^i\, t^i.$$

Step B: Expand $(a_y + td_y)^n$

Similarly:

$$(a_y + td_y)^n = \sum_{j=0}^{n} \binom{n}{j} a_y^{n-j} d_y^j \, t^j.$$

Step C: Multiply and collect powers of t

Multiply the two sums:

$$(a_x + td_x)^m (a_y + td_y)^n = \sum_{i=0}^{m} \sum_{j=0}^{n} \binom{m}{i} \binom{n}{j} a_x^{m-i} a_y^{n-j} d_x^i d_y^j t^{i+j}.$$

Step D: Integrate term-by-term

The integral of t^{i+j} is:

$$\int_0^1 t^{i+j} \, dt = \frac{1}{i+j+1}.$$

So:

$$C_{mn}^{\text{seg,geo}} = L \sum_{i=0}^{m} \sum_{j=0}^{n} \binom{m}{i} \binom{n}{j} a_x^{m-i} a_y^{n-j} d_x^i d_y^j \frac{1}{i+j+1}.$$

$$\boxed{C_{mn}^{\text{seg,geo}} = L \sum_{i=0}^{m} \sum_{j=0}^{n} \binom{m}{i} \binom{n}{j} a_x^{m-i} a_y^{n-j} d_x^i d_y^j \frac{1}{i+j+1}.} \tag{12}$$

Sanity checks.

- If $m = n = 0$, then $\phi = 1$, and the formula gives $C_{00} = L$, the segment length. That matches the idea of integrating 1 over the segment.
- If the segment is very short ($L \to 0$), the moment scales like L, which is reasonable because a segment is a 1D object.

6.3 Segment Fourier moments: step-by-step to a closed form

Now:

$$\phi_{mn}^{\text{F}}(x, y) = e^{-j\pi(mx+ny)}.$$

Plug into (11):

$$C_{mn}^{\text{seg,F}} = L \int_0^1 e^{-j\pi(m(a_x+td_x)+n(a_y+td_y))} \, dt.$$

Step A: Separate constant and variable parts

Expand the linear expression:

$$m(a_x + td_x) + n(a_y + td_y) = (ma_x + na_y) + t(md_x + nd_y).$$

Step B: Factor out the constant exponential

So:

$$e^{-j\pi[(ma_x+na_y)+t(md_x+nd_y)]} = e^{-j\pi(ma_x+na_y)} \cdot e^{-j\pi t(md_x+nd_y)}.$$

Thus:

$$C_{mn}^{\text{seg,F}} = Le^{-j\pi(ma_x+na_y)} \int_0^1 e^{-j\pi t(md_x+nd_y)}\, dt.$$

Step C: Evaluate the exponential integral

Let:

$$\alpha = \pi(md_x + nd_y).$$

Then the integral is:

$$\int_0^1 e^{-j\alpha t}\, dt.$$

This is a standard calculus result:

$$\int e^{-j\alpha t}\, dt = \frac{e^{-j\alpha t}}{-j\alpha}.$$

Evaluate from 0 to 1:

$$\int_0^1 e^{-j\alpha t}\, dt = \frac{e^{-j\alpha t}}{-j\alpha}\bigg|_0^1 = \frac{e^{-j\alpha} - 1}{-j\alpha} = \frac{1 - e^{-j\alpha}}{j\alpha}.$$

So when $\alpha \neq 0$:

$$\int_0^1 e^{-j\alpha t}\, dt = \frac{1 - e^{-j\alpha}}{j\alpha}.$$

When $\alpha = 0$, the integrand is $e^0 = 1$, so the integral is 1.

Step D: Final formula

Substitute back:

$$\boxed{C_{mn}^{\text{seg,F}} = Le^{-j\pi(ma_x+na_y)} \begin{cases} \dfrac{1 - e^{-j\pi(md_x+nd_y)}}{j\pi(md_x + nd_y)}, & md_x + nd_y \neq 0, \\ 1, & md_x + nd_y = 0. \end{cases}} \tag{13}$$

Interpretation. The term $\dfrac{1 - e^{-j\alpha}}{j\alpha}$ is essentially a complex-valued "sinc-like" function. It becomes small when $|\alpha|$ is large, meaning the segment averages out high-frequency oscillations.

6.4 Segment Legendre moments: step-by-step to a finite sum (no integrals left)

Now:

$$\phi^{\mathrm{L}}_{mn}(x,y) = P_m(x)P_n(y).$$

Start from (11):

$$C^{\mathrm{seg,L}}_{mn} = L\int_0^1 P_m(a_x + td_x)\,P_n(a_y + td_y)\,dt.$$

The strategy is the same as geometric moments:

Expand everything into powers of t, then integrate t^r term-by-term.

Step A: Expand P_m and P_n into monomials

Using (5):

$$P_m(x) = \sum_{k=0}^{\lfloor m/2\rfloor} A_{m,k}x^{m-2k}, \qquad P_n(y) = \sum_{\ell=0}^{\lfloor n/2\rfloor} A_{n,\ell}y^{n-2\ell}.$$

So:

$$P_m(x(t))P_n(y(t)) = \sum_k\sum_\ell A_{m,k}A_{n,\ell}\,(x(t))^p(y(t))^q,$$

where

$$p = m-2k, \qquad q = n-2\ell.$$

Then:

$$C^{\mathrm{seg,L}}_{mn} = L\sum_k\sum_\ell A_{m,k}A_{n,\ell}\int_0^1 (x(t))^p(y(t))^q\,dt.$$

Now we just need a closed form for the integral of $(x(t))^p(y(t))^q$.

Step B: Expand $(x(t))^p$ and $(y(t))^q$ using binomials

Recall:

$$x(t) = a_x + td_x, \qquad y(t) = a_y + td_y.$$

Expand:

$$(x(t))^p = (a_x + td_x)^p = \sum_{i=0}^{p}\binom{p}{i}a_x^{p-i}d_x^i t^i,$$

$$(y(t))^q = (a_y + td_y)^q = \sum_{j=0}^{q}\binom{q}{j}a_y^{q-j}d_y^j t^j.$$

Multiply:

$$(x(t))^p(y(t))^q = \sum_{i=0}^{p}\sum_{j=0}^{q}\binom{p}{i}\binom{q}{j}a_x^{p-i}a_y^{q-j}d_x^i d_y^j t^{i+j}.$$

Step C: Integrate term-by-term

$$\int_0^1 t^{i+j}\,dt = \frac{1}{i+j+1}.$$

So:

$$\int_0^1 (x(t))^p (y(t))^q\,dt = \sum_{i=0}^{p}\sum_{j=0}^{q}\binom{p}{i}\binom{q}{j} a_x^{p-i} a_y^{q-j} d_x^i d_y^j \frac{1}{i+j+1}.$$

Step D: Substitute back into the Legendre sum

Therefore:

$$\boxed{C_{mn}^{\text{seg,L}} = L\sum_{k=0}^{\lfloor m/2\rfloor}\sum_{\ell=0}^{\lfloor n/2\rfloor} A_{m,k}A_{n,\ell}\left[\sum_{i=0}^{m-2k}\sum_{j=0}^{n-2\ell}\binom{m-2k}{i}\binom{n-2\ell}{j} a_x^{(m-2k)-i} a_y^{(n-2\ell)-j} d_x^i d_y^j \frac{1}{i+j+1}\right].} \tag{14}$$

No integrals remain. Every sum is finite.

Sanity check. If $m = n = 0$, then $P_0(x) = 1$ and the moment reduces to the segment length L. The formula above does that (only $k = \ell = 0$ terms exist, and only $i = j = 0$ term exists).

7 Case group 3: Polygon moments via Green's theorem (very detailed)

Polygon moments are harder because they are *area integrals*:

$$C_{mn}^{\text{poly}} = \iint_\Omega \phi_{mn}(x,y)\,dx\,dy.$$

A direct approach would require integrating over a complicated polygon region. Instead we use a key vector calculus idea: convert area integrals into boundary integrals.

7.1 Green's theorem: the version we use

Green's theorem (one common form) says:

$$\iint_\Omega \left(\frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y}\right) dx\,dy = \oint_{\partial\Omega} (F_x n_x + F_y n_y)\,ds = \oint_{\partial\Omega} \mathbf{F}\cdot\mathbf{n}\,ds. \tag{15}$$

Here:

- Ω is a region (our polygon interior),
- $\partial\Omega$ is its boundary (the polygon edges),
- $\mathbf{F} = (F_x, F_y)$ is a vector field,
- $\mathbf{n} = (n_x, n_y)$ is the outward unit normal along the boundary,
- ds is arclength along the boundary.

The key trick. If we can choose $\mathbf{F}$ so that

$$\nabla \cdot \mathbf{F} = \frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y} = \phi_{mn}(x, y),$$

then the left side becomes the moment we want:

$$\iint_\Omega \phi_{mn}(x, y)\, dx\, dy = C_{mn}^{\text{poly}}.$$

And Green's theorem tells us:

$$C_{mn}^{\text{poly}} = \oint_{\partial\Omega} \mathbf{F} \cdot \mathbf{n}\, ds.$$

So the whole problem becomes:

Find a vector field whose divergence equals the basis function.

7.2 How a polygon boundary integral becomes a sum over edges

A polygon boundary $\partial\Omega$ is a sequence of straight edges. Suppose the vertices in counterclockwise order are

$$v_1 \to v_2 \to \cdots \to v_E \to v_1.$$

Then the boundary integral splits:

$$\oint_{\partial\Omega} (\cdot)\, ds = \sum_{e=1}^{E} \int_{\text{edge } e} (\cdot)\, ds.$$

So if we can compute the integral on a single edge, we can sum them.

7.3 Parameterizing one edge and its outward normal (carefully)

Consider an edge from $a = (x_0, y_0)$ to $b = (x_1, y_1)$. Define:

$$\Delta x = x_1 - x_0, \qquad \Delta y = y_1 - y_0, \qquad L = \sqrt{\Delta x^2 + \Delta y^2}.$$

Parameterize the edge by $t \in [0, 1]$:

$$x(t) = x_0 + t\Delta x, \qquad y(t) = y_0 + t\Delta y.$$

The tangent vector is $(\Delta x, \Delta y)$. The unit tangent is:

$$\mathbf{t} = \frac{1}{L}(\Delta x, \Delta y).$$

A perpendicular vector is $(\Delta y, -\Delta x)$. For a counterclockwise boundary, this points outward (right-hand rule in 2D). So the outward unit normal is:

$$\boxed{\mathbf{n} = (n_x, n_y) = \frac{1}{L}(\Delta y, -\Delta x).} \tag{16}$$

Also, arclength satisfies $ds = L\, dt$.

This is exactly what code will use later: for each edge, compute $\Delta x, \Delta y, L, n_x, n_y$.

7.4 Polygon geometric moments: very detailed derivation to an edge formula

We want the geometric polygon moment:

$$C_{mn}^{\text{poly,geo}} = \iint_{\Omega} x^m y^n \, dx \, dy.$$

Step 1: choose a vector field with divergence $x^m y^n$

We want $\nabla \cdot \mathbf{F} = x^m y^n$. A simple idea is to put everything into F_x and make $F_y = 0$.

Try:

$$F_x(x, y) = \frac{x^{m+1} y^n}{m+1}, \qquad F_y(x, y) = 0.$$

Compute divergence:

$$\nabla \cdot \mathbf{F} = \frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y} = \frac{\partial}{\partial x} \left(\frac{x^{m+1} y^n}{m+1} \right) + 0 = x^m y^n.$$

Great — it matches exactly.

Step 2: apply Green's theorem

By (15):

$$\iint_{\Omega} x^m y^n \, dx \, dy = \oint_{\partial\Omega} (F_x n_x + F_y n_y) \, ds.$$

But $F_y = 0$, so:

$$C_{mn}^{\text{poly,geo}} = \oint_{\partial\Omega} F_x n_x \, ds = \oint_{\partial\Omega} \frac{x^{m+1} y^n}{m+1} \, n_x \, ds.$$

$$\boxed{C_{mn}^{\text{poly,geo}} = \oint_{\partial\Omega} \frac{x^{m+1} y^n}{m+1} \, n_x \, ds.} \tag{17}$$

Step 3: convert boundary integral to a sum over edges

Split into edges $e = 1, \ldots, E$:

$$C_{mn}^{\text{poly,geo}} = \sum_{e=1}^{E} \int_{\text{edge } e} \frac{x^{m+1} y^n}{m+1} \, n_x \, ds.$$

On a given edge, n_x is constant (since the edge direction is constant), so we can treat it as a constant factor.

Step 4: parameterize one edge and reduce to a 1D integral

On one edge $a \to b$:

$$x(t) = x_0 + t\Delta x, \quad y(t) = y_0 + t\Delta y, \quad ds = L \, dt.$$

So the edge contribution is:

$$C_{mn,e}^{\text{geo}} = \int_0^1 \frac{(x(t))^{m+1} (y(t))^n}{m+1} \, n_x \, L \, dt.$$

This is now a 1D integral of a polynomial in t.

Step 5: expand the polynomial and integrate term-by-term

We expand:

$$(x(t))^{m+1} = (x_0 + t\Delta x)^{m+1} = \sum_{i=0}^{m+1} \binom{m+1}{i} x_0^{m+1-i} (\Delta x)^i t^i,$$

$$(y(t))^n = (y_0 + t\Delta y)^n = \sum_{j=0}^{n} \binom{n}{j} y_0^{n-j} (\Delta y)^j t^j.$$

Multiply:

$$(x(t))^{m+1}(y(t))^n = \sum_{i=0}^{m+1} \sum_{j=0}^{n} \binom{m+1}{i} \binom{n}{j} x_0^{m+1-i} y_0^{n-j} (\Delta x)^i (\Delta y)^j t^{i+j}.$$

Integrate:

$$\int_0^1 t^{i+j}\, dt = \frac{1}{i+j+1}.$$

So the edge contribution is:

$$C_{mn,e}^{\text{geo}} = \frac{n_x L}{m+1} \sum_{i=0}^{m+1} \sum_{j=0}^{n} \binom{m+1}{i} \binom{n}{j} x_0^{m+1-i} y_0^{n-j} (\Delta x)^i (\Delta y)^j \frac{1}{i+j+1}.$$

Finally, sum over edges.

$$\boxed{C_{mn}^{\text{poly,geo}} = \sum_{e=1}^{E} \frac{n_{x,e} L_e}{m+1} \sum_{i=0}^{m+1} \sum_{j=0}^{n} \binom{m+1}{i} \binom{n}{j} x_{0,e}^{m+1-i} y_{0,e}^{n-j} (\Delta x_e)^i (\Delta y_e)^j \frac{1}{i+j+1}.} \tag{18}$$

This is an explicit, finite closed form.

7.5 Polygon Fourier moments: very detailed derivation to an edge formula

We want:

$$C_{mn}^{\text{poly,F}} = \iint_\Omega e^{-j\pi(mx+ny)}\, dx\, dy.$$

Step 1: choose a vector field whose divergence equals the exponential

We want $\nabla \cdot \mathbf{F} = e^{-j\pi(mx+ny)}$.

A good guess is to make $\mathbf{F}$ proportional to the exponential itself, because derivatives of exponentials produce exponentials again.

Let:

$$\mathbf{F}(x,y) = \gamma \binom{m}{n} e^{-j\pi(mx+ny)}.$$

Compute divergence:

$$\nabla \cdot \mathbf{F} = \frac{\partial}{\partial x}\left(\gamma m e^{-j\pi(mx+ny)}\right) + \frac{\partial}{\partial y}\left(\gamma n e^{-j\pi(mx+ny)}\right).$$

Since γm is constant:

$$\frac{\partial}{\partial x} e^{-j\pi(mx+ny)} = (-j\pi m) e^{-j\pi(mx+ny)}.$$

Similarly:

$$\frac{\partial}{\partial y} e^{-j\pi(mx+ny)} = (-j\pi n) e^{-j\pi(mx+ny)}.$$

So:

$$\nabla \cdot \mathbf{F} = \gamma m(-j\pi m) e^{-j\pi(mx+ny)} + \gamma n(-j\pi n) e^{-j\pi(mx+ny)} = \gamma(-j\pi)(m^2+n^2) e^{-j\pi(mx+ny)}.$$

We want this to equal $e^{-j\pi(mx+ny)}$. Therefore choose:

$$\gamma = \frac{1}{-j\pi(m^2+n^2)}.$$

So we take:

$$\mathbf{F}(x,y) = \frac{1}{-j\pi(m^2+n^2)} \begin{pmatrix} m \\ n \end{pmatrix} e^{-j\pi(mx+ny)}.$$

Step 2: apply Green's theorem

Green's theorem gives:

$$C_{mn}^{\text{poly,F}} = \oint_{\partial\Omega} \mathbf{F} \cdot \mathbf{n}\, ds.$$

Compute the dot product:

$$\mathbf{F} \cdot \mathbf{n} = \frac{1}{-j\pi(m^2+n^2)} \left(mn_x + nn_y\right) e^{-j\pi(mx+ny)}.$$

So:

$$\boxed{C_{mn}^{\text{poly,F}} = \frac{1}{-j\pi(m^2+n^2)} \oint_{\partial\Omega} \left(mn_x + nn_y\right) e^{-j\pi(mx+ny)}\, ds.} \tag{19}$$

Step 3: reduce to an explicit edge integral

Split the boundary into edges:

$$C_{mn}^{\text{poly,F}} = \frac{1}{-j\pi(m^2+n^2)} \sum_{e=1}^{E} (mn_{x,e} + nn_{y,e}) \int_{\text{edge } e} e^{-j\pi(mx+ny)}\, ds.$$

On one edge $a \to b$, parameterize:

$$x(t) = x_0 + t\Delta x, \quad y(t) = y_0 + t\Delta y, \quad ds = L\, dt.$$

Then:

$$\int_{\text{edge}} e^{-j\pi(mx+ny)}\, ds = L \int_0^1 e^{-j\pi(mx(t)+ny(t))}\, dt.$$

Compute the phase:

$$mx(t) + ny(t) = m(x_0 + t\Delta x) + n(y_0 + t\Delta y) = (mx_0 + ny_0) + t(m\Delta x + n\Delta y).$$

Define:

$$\beta = \pi(mx_0 + ny_0), \qquad \alpha = \pi(m\Delta x + n\Delta y).$$

Then:

$$e^{-j\pi(mx(t)+ny(t))} = e^{-j\beta}e^{-j\alpha t}.$$

So:

$$L\int_0^1 e^{-j\pi(mx(t)+ny(t))}\,dt = Le^{-j\beta}\int_0^1 e^{-j\alpha t}\,dt.$$

And we already evaluated:

$$\int_0^1 e^{-j\alpha t}\,dt = \begin{cases} \frac{1-e^{-j\alpha}}{j\alpha}, & \alpha \neq 0, \\ 1, & \alpha = 0. \end{cases}$$

So each edge contributes a simple closed form, and the polygon moment is a finite sum over edges.

$$\boxed{C_{mn}^{\text{poly,F}} = \sum_{e=1}^{E} \frac{L_e(mn_{x,e} + nn_{y,e})}{-j\pi(m^2+n^2)}\, e^{-j\beta_e} \begin{cases} \dfrac{1-e^{-j\alpha_e}}{j\alpha_e}, & \alpha_e \neq 0, \\ 1, & \alpha_e = 0, \end{cases}} \tag{20}$$

where $\beta_e = \pi(mx_{0,e} + ny_{0,e})$ and $\alpha_e = \pi(m\Delta x_e + n\Delta y_e)$.

7.6 Polygon Legendre moments: very detailed reduction to geometric moments

We want:

$$C_{mn}^{\text{poly,L}} = \iint_\Omega P_m(x)P_n(y)\,dx\,dy.$$

The key is that $P_m(x)$ is a polynomial. So $P_m(x)P_n(y)$ is a polynomial in x and y.

Step 1: expand each Legendre polynomial into monomials

Using (5):

$$P_m(x) = \sum_{k=0}^{\lfloor m/2\rfloor} A_{m,k}\,x^{m-2k}, \qquad P_n(y) = \sum_{\ell=0}^{\lfloor n/2\rfloor} A_{n,\ell}\,y^{n-2\ell}.$$

Multiply:

$$P_m(x)P_n(y) = \sum_k \sum_\ell A_{m,k}A_{n,\ell}\,x^{m-2k}y^{n-2\ell}.$$

Step 2: integrate term-by-term over the polygon

Then:

$$C_{mn}^{\text{poly,L}} = \iint_\Omega \left(\sum_k \sum_\ell A_{m,k}A_{n,\ell}\,x^{m-2k}y^{n-2\ell}\right) dx\,dy.$$

Swap sum and integral (linearity):

$$C_{mn}^{\text{poly,L}} = \sum_k \sum_\ell A_{m,k}A_{n,\ell} \iint_\Omega x^{m-2k}y^{n-2\ell}\,dx\,dy.$$

But the remaining integrals are exactly geometric polygon moments:

$$\iint_\Omega x^p y^q\,dx\,dy = C_{pq}^{\text{poly,geo}}.$$

So:

$$\boxed{C_{mn}^{\text{poly,L}} = \sum_{k=0}^{\lfloor m/2 \rfloor} \sum_{\ell=0}^{\lfloor n/2 \rfloor} A_{m,k} A_{n,\ell}\, C_{(m-2k),(n-2\ell)}^{\text{poly,geo}}.} \tag{21}$$

And each $C_{pq}^{\text{poly,geo}}$ is computed by the edge closed form (18). So Legendre polygon moments are also closed form.

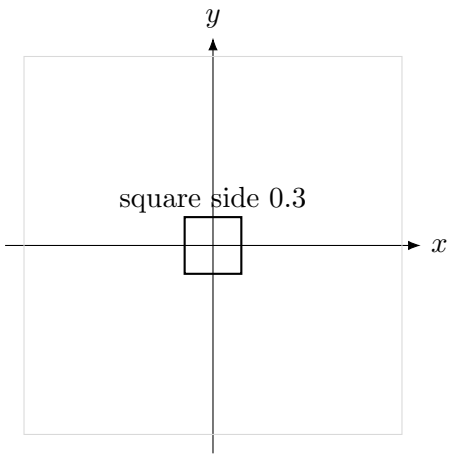
8 Two polygon examples (conceptual setup for Chunk 2)

Chunk 2 will compute both examples numerically and show edge tables. Here, we set them up and explain why the square simplifies.

8.1 Example A: Square centered at origin, side length 0.3

Let $a = 0.15$. Vertices CCW:

$$(-a, -a) \to (a, -a) \to (a, a) \to (-a, a).$$



Why the square "collapses" to easier formulas. Because this square is axis-aligned, the region is simply:

$$\Omega = [-a, a] \times [-a, a].$$

So area integrals separate:

$$\iint_{\Omega} x^m y^n \, dx \, dy = \left(\int_{-a}^{a} x^m dx \right) \left(\int_{-a}^{a} y^n dy \right).$$

This makes it easy to check results.

Similarly, Legendre moments separate:

$$\iint_{\Omega} P_m(x) P_n(y) \, dx \, dy = \left(\int_{-a}^{a} P_m(x) \, dx \right) \left(\int_{-a}^{a} P_n(y) \, dy \right).$$

For Fourier:

$$\iint_{\Omega} e^{-j\pi(mx+ny)} \, dx \, dy = \left(\int_{-a}^{a} e^{-j\pi mx} \, dx \right) \left(\int_{-a}^{a} e^{-j\pi ny} \, dy \right),$$

again a product of two 1D sinc-like terms.

Why we still use boundary formulas. Even though the square has a simpler "rectangle integral" form, we will compute it using boundary sums (Green's theorem) so the method stays consistent for general polygons.

8.2 Example B: A less symmetric polygon

We also use an asymmetric convex quadrilateral:

$$(-0.30, -0.20) \to (0.35, -0.10) \to (0.25, 0.40) \to (-0.20, 0.30).$$

This avoids symmetry cancellations, so moments like C_{21} will generally be nonzero.

Chunk 2 will compute moments for both $(m, n) = (2, 1)$ (shows symmetry zero for square) and $(m, n) = (2, 2)$ (shows nonzero), for both polygons, with per-edge tables.

9 Summary of the closed forms derived in Chunk 1

We derived closed forms for:

- Point moments: (10)
- Segment geometric: (12)
- Segment Fourier: (13)
- Segment Legendre: (14)
- Polygon geometric (boundary + edge): (17), (18)
- Polygon Fourier (boundary + edge): (19), (20)
- Polygon Legendre: (21)

Chunk 2 will add:

- all numerical examples (all 9 cases),
- two polygon examples (square + asymmetric),
- per-edge tables (geometric, Fourier, Legendre),
- modular Python code reproducing every number shown.

10 What this chunk contains

This chunk provides:

- numerical examples for all $3 \times 3 = 9$ cases:

$$\{\text{point, segment, polygon}\} \times \{\text{geometric, Fourier, Legendre}\},$$

- two index pairs:

$$(m, n) \in \{(2, 1), (2, 2)\},$$

- two polygon examples:
  - **Square centered at origin**, side length 0.3 (so we can show simplifications),
  - **Less symmetric quadrilateral** (so results are typically nonzero).
- per-edge tables for **all three** polygon moment types (geometric, Fourier, Legendre),
- modular Python code that reproduces every number shown and prints the same tables.

Notation for complex numbers. We write complex values as $a + bj$, where $j = \sqrt{-1}$.

11 Shared example inputs

11.1 Two index pairs

We compute everything for:

$$(m, n) = (2, 1) \quad \text{and} \quad (m, n) = (2, 2).$$

11.2 Point example

Point location:

$$(x_0, y_0) = (0.7, -0.2).$$

11.3 Line segment example

Segment endpoints:

$$a = (-0.6, -0.1), \qquad b = (0.2, 0.7).$$

11.4 Polygon example A: square centered at origin, side length 0.3

Half side length $a = 0.15$. CCW vertices:

$$(-0.15, -0.15) \to (0.15, -0.15) \to (0.15, 0.15) \to (-0.15, 0.15).$$

11.5 Polygon example B: less symmetric quadrilateral

CCW vertices:

$$(-0.30, -0.20) \to (0.35, -0.10) \to (0.25, 0.40) \to (-0.20, 0.30).$$

12 Point results (3 cases)

12.1 For $(m, n) = (2, 1)$

$$C_{21}^{\mathrm{pt,geo}} = -0.098000000, \quad C_{21}^{\mathrm{pt,F}} = -0.809016994 + 0.587785252\, j, \quad C_{21}^{\mathrm{pt,L}} = -0.047000000.$$

12.2 For $(m, n) = (2, 2)$

$$C_{22}^{\mathrm{pt,geo}} = 0.019600000, \quad C_{22}^{\mathrm{pt,F}} = -1.000000000 - 0.000000000\, j, \quad C_{22}^{\mathrm{pt,L}} = -0.103400000.$$

13 Line segment results (3 cases)

13.1 For $(m, n) = (2, 1)$

$$C_{21}^{\text{seg,geo}} = 0.007542472, \quad C_{21}^{\text{seg,F}} = -0.167764040 - 0.054509841\, j, \quad C_{21}^{\text{seg,L}} = -0.158391919.$$

13.2 For $(m, n) = (2, 2)$

$$C_{22}^{\text{seg,geo}} = 0.003228178, \quad C_{22}^{\text{seg,F}} = -0.173180544 + 0.125823030\, j, \quad C_{22}^{\text{seg,L}} = 0.089287787.$$

14 Polygon A: square centered at origin, side 0.3

14.1 Why the square "collapses" to simpler formulas

Because the square is axis-aligned, its interior is simply:

$$\Omega = [-a, a] \times [-a, a], \quad a = 0.15.$$

So geometric moments separate:

$$\iint_\Omega x^m y^n \, dx \, dy = \left(\int_{-a}^{a} x^m \, dx \right) \left(\int_{-a}^{a} y^n \, dy \right).$$

Legendre moments also separate:

$$\iint_\Omega P_m(x) P_n(y) \, dx \, dy = \left(\int_{-a}^{a} P_m(x) \, dx \right) \left(\int_{-a}^{a} P_n(y) \, dy \right).$$

Fourier moments separate:

$$\iint_\Omega e^{-j\pi(mx+ny)} \, dx \, dy = \left(\int_{-a}^{a} e^{-j\pi mx} \, dx \right) \left(\int_{-a}^{a} e^{-j\pi ny} \, dy \right).$$

We will still compute everything using boundary/edge formulas (for consistency), but the square provides a strong check.

14.2 Square: geometric and Legendre "collapse" checks

For $(m, n) = (2, 1)$, the integrand is odd in y, so:

$$C_{21}^{\text{square,geo}} = 0, \qquad C_{21}^{\text{square,L}} = 0.$$

For $(m, n) = (2, 2)$,

$$\int_{-a}^{a} x^2 \, dx = \frac{2a^3}{3} = \frac{2(0.15)^3}{3} = 0.00225.$$

So

$$C_{22}^{\text{square,geo}} = (0.00225)^2 = 0.0000050625.$$

Also $P_2(x) = \frac{1}{2}(3x^2 - 1)$, so:

$$\int_{-a}^{a} P_2(x) \, dx = \int_{-a}^{a} \tfrac{1}{2}(3x^2 - 1) \, dx = \tfrac{1}{2} \left(3 \cdot \frac{2a^3}{3} - 2a \right) = a^3 - a = 0.003375 - 0.15 = -0.146625.$$

Thus:

$$C_{22}^{\text{square,L}} = (-0.146625)^2 = 0.021498890625.$$

These match the boundary/edge computations below.

14.3 Square totals (boundary computation)

For $(m, n) = (2, 1)$

$$C_{21}^{\text{square,geo}} = 0.000000000, \quad C_{21}^{\text{square,F}} = 0.074427711 + 0.000000000\, j, \quad C_{21}^{\text{square,L}} = 0.000000000.$$

For $(m, n) = (2, 2)$

$$C_{22}^{\text{square,geo}} = 0.0000050625, \quad C_{22}^{\text{square,F}} = 0.066315576 + 0.000000000\, j, \quad C_{22}^{\text{square,L}} = 0.021498890625.$$

14.4 Square per-edge tables

Each table row includes: edge geometry and three per-edge contributions.

Square, $(m, n) = (2, 1)$**.**

Edge	$(\Delta x, \Delta y)$	L	(n_x, n_y)	Geo contrib	Fourier contrib	Legendre contrib
1	$(0.300, 0.000)$	0.300	$(0.000, -1.000)$	0.000000000	$0.007442771 - 0.014607261j$	0.000000000
2	$(0.000, 0.300)$	0.300	$(1.000, -0.000)$	0.000000000	$0.029771084 + 0.021629959j$	0.000000000
3	$(-0.300, 0.000)$	0.300	$(0.000, 1.000)$	0.000000000	$0.007442771 + 0.014607261j$	0.000000000
4	$(0.000, -0.300)$	0.300	$(-1.000, -0.000)$	0.000000000	$0.029771084 - 0.021629959j$	0.000000000

Square, $(m, n) = (2, 2)$**.**

Edge	$(\Delta x, \Delta y)$	L	(n_x, n_y)	Geo contrib	Fourier contrib	Legendre contrib
1	$(0.300, 0.000)$	0.300	$(0.000, -1.000)$	0.000000000	$0.016578894 - 0.012045271j$	0.000000000
2	$(0.000, 0.300)$	0.300	$(1.000, -0.000)$	0.000002531	$0.016578894 + 0.012045271j$	0.010749445
3	$(-0.300, 0.000)$	0.300	$(0.000, 1.000)$	0.000000000	$0.016578894 + 0.012045271j$	0.000000000
4	$(0.000, -0.300)$	0.300	$(-1.000, -0.000)$	0.000002531	$0.016578894 - 0.012045271j$	0.010749445

15 Polygon B: less symmetric quadrilateral

15.1 Quadrilateral totals

For $(m, n) = (2, 1)$

$$C_{21}^{\text{quad,geo}} = 0.000488896, \quad C_{21}^{\text{quad,F}} = 0.106454907 - 0.061860239\, j, \quad C_{21}^{\text{quad,L}} = -0.011016656.$$

For $(m, n) = (2, 2)$

$$C_{22}^{\text{quad,geo}} = 0.000188216, \quad C_{22}^{\text{quad,F}} = 0.046986391 - 0.059875625\, j, \quad C_{22}^{\text{quad,L}} = 0.057707861.$$

15.2 Quadrilateral per-edge tables

Quadrilateral, $(m, n) = (2, 1)$**.**

Edge	$(\Delta x, \Delta y)$	L	(n_x, n_y)	Geo contrib	Fourier contrib	Legendre contrib
1	$(0.650, 0.100)$	0.658	$(0.152, -0.988)$	-0.000001500	$0.003256752 - 0.010023253j$	-0.000085583
2	$(-0.100, 0.500)$	0.510	$(0.981, 0.196)$	0.000505208	$0.047704950 - 0.047704950j$	-0.009450521
3	$(-0.450, -0.100)$	0.461	$(-0.217, 0.976)$	-0.000018979	$0.009636217 + 0.003130997j$	0.000596531
4	$(-0.100, -0.500)$	0.510	$(-0.981, 0.196)$	0.000004167	$0.045856988 - 0.007263033j$	-0.002077083

Quadrilateral, $(m, n) = (2, 2)$**.**

Edge	$(\Delta x, \Delta y)$	L	(n_x, n_y)	Geo contrib	Fourier contrib	Legendre contrib
1	$(0.650, 0.100)$	0.658	$(0.152, -0.988)$	-0.000001410	$0.009287775 - 0.009287775j$	0.000633547
2	$(-0.100, 0.500)$	0.510	$(0.981, 0.196)$	0.000145938	$0.011166581 - 0.034367202j$	0.029953359
3	$(-0.450, -0.100)$	0.461	$(-0.217, 0.976)$	-0.000008117	$0.011257723 - 0.011257723j$	-0.000183108
4	$(-0.100, -0.500)$	0.510	$(-0.981, 0.196)$	0.000051806	$0.015274312 - 0.004962925j$	0.027304063

16 Python code (modular, reproduces all numbers and prints tables)

This script prints:

- point + segment outputs for both $(2, 1)$ and $(2, 2)$,
- square polygon edge tables and totals for both indices,
- quadrilateral polygon edge tables and totals for both indices,
- including Legendre **per-edge** contributions computed via monomial expansion.

```
import math
import cmath
from typing import Dict, List, Tuple

# ============================================================
# 1) Legendre coefficients: P_m(x)=sum_k c[k] x^k
# ============================================================

def legendre_coeffs(m: int) -> Dict[int, float]:
    """
    P_m(x)= sum_{k=0..floor(m/2)} (-1)^k * (2m-2k)! / (2^m k! (m-k)! (m-2k)!) * x^{m-2k}.
    Returns dict {power: coefficient}.
    """
    c: Dict[int, float] = {}
    for k in range(0, m//2 + 1):
```

```
        deg = m - 2*k
        num = math.factorial(2*m - 2*k)
        den = (2**m) * math.factorial(k) * math.factorial(m-k) * math.
           factorial(m-2*k)
        coef = ((-1)**k) * (num / den)
        c[deg] = c.get(deg, 0.0) + coef
    return c

def poly_eval(coeffs: Dict[int, float], x: float) -> float:
    return sum(ck * (x**k) for k, ck in coeffs.items())

# ============================================================
# 2) Point moments (3 cases)
# ============================================================

def moment_point_geo(x0: float, y0: float, m: int, n: int) -> float:
    return (x0**m) * (y0**n)

def moment_point_fourier(x0: float, y0: float, m: int, n: int) -> complex:
    return cmath.exp(-1j * math.pi * (m*x0 + n*y0))

def moment_point_legendre(x0: float, y0: float, m: int, n: int) -> float:
    return poly_eval(legendre_coeffs(m), x0) * poly_eval(legendre_coeffs(n
       ), y0)

# ============================================================
# 3) Segment moments (3 cases)
# ============================================================

def seg_params(a: Tuple[float,float], b: Tuple[float,float]) -> Tuple[
   float,float,float]:
    dx = b[0] - a[0]
    dy = b[1] - a[1]
    L = math.hypot(dx, dy)
    return dx, dy, L

def moment_seg_geo(a: Tuple[float,float], b: Tuple[float,float], m: int, n
   : int) -> float:
    ax, ay = a
    dx, dy, L = seg_params(a, b)
    total = 0.0
    for i in range(m+1):
        for j in range(n+1):
            total += (math.comb(m, i) * math.comb(n, j) *
                      (ax**(m-i)) * (ay**(n-j)) *
                      (dx**i) * (dy**j) /
                      (i+j+1))
    return L * total

def moment_seg_fourier(a: Tuple[float,float], b: Tuple[float,float], m:
   int, n: int) -> complex:
    ax, ay = a
    dx, dy, L = seg_params(a, b)
    kdot_a = m*ax + n*ay
```

```
    kdot_d = m*dx + n*dy
    phase0 = cmath.exp(-1j * math.pi * kdot_a)
    if abs(kdot_d) < 1e-14:
        return L * phase0
    num = 1.0 - cmath.exp(-1j * math.pi * kdot_d)
    den = 1j * math.pi * kdot_d
    return L * phase0 * (num / den)

def moment_seg_legendre(a: Tuple[float,float], b: Tuple[float,float], m: int, n: int) -> float:
    ax, ay = a
    dx, dy, L = seg_params(a, b)

    Pm = legendre_coeffs(m)
    Pn = legendre_coeffs(n)

    # Substitute x(t)=ax + t dx into polynomial Pm(x) to produce polynomial in t.
    def subst_poly(poly_x: Dict[int,float], a0: float, d0: float) -> Dict[int,float]:
        out: Dict[int,float] = {}
        for k, ck in poly_x.items():
            for i in range(k+1):
                out[i] = out.get(i, 0.0) + ck * math.comb(k, i) * (a0**(k-i)) * (d0**i)
        return out

    pm_t = subst_poly(Pm, ax, dx)
    pn_t = subst_poly(Pn, ay, dy)

    # Multiply polynomials in t
    prod: Dict[int,float] = {}
    for i, ci in pm_t.items():
        for j, cj in pn_t.items():
            prod[i+j] = prod.get(i+j, 0.0) + ci*cj

    # Integrate t^p from 0 to 1: 1/(p+1)
    integral = sum(cp/(p+1) for p, cp in prod.items())
    return L * integral

# ============================================================
# 4) Polygon boundary machinery
# ============================================================

def edge_params_ccw(a: Tuple[float,float], b: Tuple[float,float]) -> Tuple[float,float,float,float,float]:
    dx = b[0] - a[0]
    dy = b[1] - a[1]
    L  = math.hypot(dx, dy)
    nx = dy / L
    ny = -dx / L
    return dx, dy, L, nx, ny

def poly_geo_edge_contrib(a: Tuple[float,float], b: Tuple[float,float], m:
```

```
    int, n: int) -> float:
    """
    Per-edge contribution to C_mn^poly,geo via boundary integral:
      C =     (x^{m+1} y^n)/(m+1) * n_x ds.
    """
    x0, y0 = a
    dx, dy, L, nx, ny = edge_params_ccw(a, b)

    integ = 0.0
    for i in range(m+2):
        for j in range(n+1):
            integ += (math.comb(m+1, i) * math.comb(n, j) *
                      (x0**(m+1-i)) * (y0**(n-j)) *
                      (dx**i) * (dy**j) /
                      (i+j+1))
    return (nx * L * integ) / (m+1)

def poly_geo_verbose(verts_ccw: List[Tuple[float,float]], m: int, n: int):
    total = 0.0
    rows = []
    for e in range(len(verts_ccw)):
        a = verts_ccw[e]
        b = verts_ccw[(e+1) % len(verts_ccw)]
        dx, dy, L, nx, ny = edge_params_ccw(a, b)
        contrib = poly_geo_edge_contrib(a, b, m, n)
        total += contrib
        rows.append((e+1, dx, dy, L, nx, ny, contrib))
    return total, rows

def poly_fourier_verbose(verts_ccw: List[Tuple[float,float]], m: int, n:
   int):
    total = 0.0 + 0.0j
    rows = []
    denom = -1j * math.pi * (m*m + n*n)
    for e in range(len(verts_ccw)):
        a = verts_ccw[e]
        b = verts_ccw[(e+1) % len(verts_ccw)]
        x0, y0 = a
        dx, dy, L, nx, ny = edge_params_ccw(a, b)

        alpha = math.pi * (m*dx + n*dy)
        beta  = math.pi * (m*x0 + n*y0)

        if abs(alpha) < 1e-12:
            I = 1.0 + 0.0j
        else:
            I = (1.0 - cmath.exp(-1j*alpha)) / (1j*alpha)

        weight  = (L * (m*nx + n*ny)) / denom
        contrib = weight * cmath.exp(-1j*beta) * I
        total  += contrib
        rows.append((e+1, dx, dy, L, nx, ny, alpha, beta, contrib))
    return total, rows

```

```
def poly_legendre_verbose(verts_ccw: List[Tuple[float,float]], m: int, n: int):
    """
    Per-edge Legendre contributions via monomial expansion:
      P_m(x)P_n(y) = sum_{k,ell} c_m[k] c_n[ell] x^k y^ell,
    and each monomial uses geometric per-edge contributions.
    """
    Pm = legendre_coeffs(m)
    Pn = legendre_coeffs(n)
    edge_contrib = [0.0 for _ in range(len(verts_ccw))]

    for k, ck in Pm.items():
        for ell, cl in Pn.items():
            w = ck * cl
            for e in range(len(verts_ccw)):
                a = verts_ccw[e]
                b = verts_ccw[(e+1) % len(verts_ccw)]
                edge_contrib[e] += w * poly_geo_edge_contrib(a, b, k, ell)

    total = sum(edge_contrib)
    rows = []
    for e in range(len(verts_ccw)):
        a = verts_ccw[e]
        b = verts_ccw[(e+1) % len(verts_ccw)]
        dx, dy, L, nx, ny = edge_params_ccw(a, b)
        rows.append((e+1, dx, dy, L, nx, ny, edge_contrib[e]))
    return total, rows

# ============================================================
# 5) Pretty printers
# ============================================================

def cfmt(z: complex, nd: int = 9) -> str:
    return f"{z.real:.{nd}f}{z.imag:+.{nd}f}j"

def ff(x: float, nd: int = 9) -> str:
    return f"{x:.{nd}f}"

def print_poly_tables(name: str, verts: List[Tuple[float,float]], m: int, n: int):
    print("\n======================================================")
    print(f"POLYGON: {name}  (m,n)=({m},{n})")
    print("verts:", verts)
    print("======================================================")

    Cg, gr = poly_geo_verbose(verts, m, n)
    print("\nGeometric total:", ff(Cg))
    print("edge | dx dy | L | nx ny | contrib")
    for (e, dx, dy, L, nx, ny, c) in gr:
        print(f"{e:>4d} | ({dx:+.3f},{dy:+.3f}) | {L:.3f} | ({nx:+.3f},{ny:+.3f}) | {ff(c)}")

    Cf, fr = poly_fourier_verbose(verts, m, n)
    print("\nFourier total:", cfmt(Cf))
```

```
    print("edge | dx dy | L | nx ny | alpha beta | contrib")
    for (e, dx, dy, L, nx, ny, alpha, beta, c) in fr:
        print(f"{e:>4d} | ({dx:+.3f},{dy:+.3f}) | {L:.3f} | ({nx:+.3f},{ny:+.3f}) | "
              f"{alpha:+.3f} {beta:+.3f} | {cfmt(c)}")

    Cl, lr = poly_legendre_verbose(verts, m, n)
    print("\nLegendre total:", ff(Cl))
    print("edge | dx dy | L | nx ny | contrib")
    for (e, dx, dy, L, nx, ny, c) in lr:
        print(f"{e:>4d} | ({dx:+.3f},{dy:+.3f}) | {L:.3f} | ({nx:+.3f},{ny:+.3f}) | {ff(c)}")

# ============================================================
# 6) Main: reproduces all examples
# ============================================================

def main():
    indices = [(2,1), (2,2)]

    # Point
    x0, y0 = 0.7, -0.2

    # Segment
    a = (-0.6, -0.1)
    b = (0.2, 0.7)

    # Polygons
    half = 0.15
    square = [(-half,-half), (half,-half), (half,half), (-half,half)]
    quad   = [(-0.30,-0.20), (0.35,-0.10), (0.25,0.40), (-0.20,0.30)]

    for (m,n) in indices:
        print("\n======================================================")
        print(f"INDEX (m,n)=({m},{n})")
        print("======================================================")

        # Point (3)
        print("\nPOINT:")
        print("  geo     :", ff(moment_point_geo(x0,y0,m,n)))
        print("  Fourier :", cfmt(moment_point_fourier(x0,y0,m,n)))
        print("  Legendre:", ff(moment_point_legendre(x0,y0,m,n)))

        # Segment (3)
        print("\nSEGMENT:")
        print("  a->b =", a, "->", b)
        print("  geo     :", ff(moment_seg_geo(a,b,m,n)))
        print("  Fourier :", cfmt(moment_seg_fourier(a,b,m,n)))
        print("  Legendre:", ff(moment_seg_legendre(a,b,m,n)))

        # Polygons (3 each) + per-edge tables
        print_poly_tables("Square centered at origin, side 0.3", square, m, n)
        print_poly_tables("Less symmetric quadrilateral", quad, m, n)
```

```
268
269 if __name__ == "__main__":
270     main()
```